task_12.py - C:/Users/josth/AppData/Local/Programs/Python/Python313/task_12.py (3.13.17)

File Edit Format Run Options Window Help

```
from decimal import Decimal, ROUND_HALF_UP

# Seasonal rate cards
def get_rate_card(season):
    if season.lower() == "april-june":
        return {
            "base": Decimal("32.00"),
            "fat_step": Decimal("0.45"),
            "snf_step": Decimal("0.30"),
            "reference_fat": Decimal("3.50"),
            "reference_snf": Decimal("8.50")
        }
    else:
        return {
            "base": Decimal("30.00"),
            "fat_step": Decimal("0.40"),
            "snf_step": Decimal("0.30"),
            "reference_fat": Decimal("3.50"),
            "reference_snf": Decimal("8.50")
        }


# Calculate milk rate using two-decimal readings
def calculate_milk_rate(fat, snf, season):
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
    card = get_rate_card(season)

    if not Decimal("3.00") <= fat <= Decimal("6.00"):
        raise ValueError("Fat must be between 3.00 and 6.00")

    if snf <= 0:
        raise ValueError("SNF must be positive")

    # Each 0.10 change is one step
    fat_steps = ((fat - card["reference_fat"]) * 10).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )
    snf_steps = ((snf - card["reference_snf"]) * 10).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP
    )

    rate = (
        card["base"]
        + fat_steps * card["fat_step"]
        + snf_steps * card["snf_step"]
```

```
            + snf_steps * card["snf_step"]
    )

    return rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)


# Calculate payment and bonus
def calculate_payment(litres, fat, snf, season, ten_day_total):
    litres = Decimal(str(litres))
    ten_day_total = Decimal(str(ten_day_total))

    if litres <= 0:
        raise ValueError("Litres must be positive")

    rate = calculate_milk_rate(fat, snf, season)
    basic_payment = litres * rate

    bonus_per_litre = Decimal("0.50") if ten_day_total > 500 else Decimal("0.00")
    bonus = litres * bonus_per_litre
    total_payment = basic_payment + bonus

    return (
        rate,
        basic_payment.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP),
        bonus.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP),
        total_payment.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    )


# Test cases
print("TASK 12: ADAPTED MILK PAYMENT SYSTEM")
print("------------------------------------")

tests = [
    (10, "3.50", "8.50", "April-June", 510),
    (10, "4.25", "8.65", "April-June", 510),
    (10, "4.25", "8.65", "July-March", 500),
    (10, "3.75", "8.55", "July-March", 501)
]

for litres, fat, snf, season, ten_day_total in tests:
    rate, basic, bonus, total = calculate_payment(
        litres, fat, snf, season, ten_day_total
    )
```

```python
# Test cases
print("TASK 12: ADAPTED MILK PAYMENT SYSTEM")
print("-------------------------------------")

tests = [
    (10, "3.50", "8.50", "April-June", 510),
    (10, "4.25", "8.65", "April-June", 510),
    (10, "4.25", "8.65", "July-March", 500),
    (10, "3.75", "8.55", "July-March", 501)
]

for litres, fat, snf, season, ten_day_total in tests:
    rate, basic, bonus, total = calculate_payment(
        litres, fat, snf, season, ten_day_total
    )

    print("\nSeason:", season)
    print("Fat:", fat, " SNF:", snf)
    print("Today's litres:", litres)
    print("Ten-day total:", ten_day_total)
    print("Milk rate: Rs.", rate)
    print("Basic payment: Rs.", basic)
    print("Bonus: Rs.", bonus)
    print("Total payment: Rs.", total)

print("\nAll tests completed.")
```